

COMP 421: Files & Databases

Lecture 9: Sorting & Aggregation

Code Review 1

Scheduling issues? See me or send email.

30 minute slot, aiming for 25 minute conversation

3 points of discussion

- 1 general question about P1 implementation (5-7 min.)

- 1 more in-depth chat about BPM design decisions (15 min.)

- 1 high-level question about B+ Tree assignment (5-8 min.)

Code Review 1

Goal is to assess literacy in more granular details of DMBS implementation

Bring a laptop so we can see your dev repo while you talk

Other presentation options:

- Running specific tests

- Stepping through code (possibly in debug mode)

- Drawing on paper

- Drawing on whiteboard

Code Review 1

No use of internet search or AI question asking. We will not try and trip you up with tricks and minute details. If you ask a reasonable question, the instructor may pull up some reference material for you.

This material will be on the website later today.

Canonical Interview Question

What is the most efficient way to sort 1 million 32-bit integers?

Time complexity?

$O(n \log n)$ (comparison-based)

$O(n)$ (radix/math)

Space complexity?

$$O(n) \approx 1M \cdot 4B = 4MB$$

In-memory Sorting

If data fits in memory, then we can use a standard sorting algorithm like Quicksort / TimSort.

→ Many online [visualization tools](#).

If data does not fit in memory, then we need to use a technique that is aware of the cost of reading and writing disk pages ...

The Justin Timberlake Problem



Interesting Question

What is the most efficient way to sort **10 billion** 32-bit integers?

Space complexity?

$$O(n) \approx 10B \cdot 4B = 40GB$$

Maybe bigger than DRAM, probably bigger than what we want in the buffer pool

Time complexity?

$O(n \log n)$ (comparison-based) $O(n)$ (radix/math)

Ignores huge constants. Number of I/O operations?

Algorithms for a Disk-Oriented DBMS

Just like it cannot assume that a table fits entirely in memory, a disk-oriented DBMS cannot assume that query results fit in memory.

We will use the buffer pool to implement algorithms that need to spill to disk.

We are also going to prefer algorithms that maximize the amount of sequential I/O.

Why Do We Need To Sort?

Relational model/SQL is unsorted.

Queries may request that tuples are sorted in a specific way (**ORDER BY**).

But even if a query does not specify an order, we may still want to sort to do other things:

- Trivial to support duplicate elimination (**DISTINCT**).
- Bulk loading sorted tuples into a B+Tree index is faster.
- Aggregations (**GROUP BY**).

Today's Agenda

Top-N Heap Sort

External Merge Sort

Aggregations

Top-N Heap Sort

If a query contains an **ORDER BY** with a **LIMIT**, then the DBMS only needs to scan the data once to find the top-N elements.

Ideal scenario for heapsort if the top-N elements fit in memory.

→ Scan data once, maintain an in-memory sorted priority queue.

```
SELECT * FROM enrolled  
ORDER BY sid ASC  
FETCH FIRST 4 ROWS  
WITH TIES
```

Original Data

3	4	6	2	9	1	4	4	8
---	---	---	---	---	---	---	---	---



Skip!

Skip and done!

Sorted Heap

1	4	6		4	4		
---	---	---	--	---	---	--	--

External Merge Sort

Divide-and-conquer algorithm that splits data into separate runs, sorts them individually, and then combines them into longer sorted runs.

Phase #1 – Sorting

→ Sort chunks of data that fit in memory and then write back the sorted chunks to a file on disk.

Phase #2 – Merging

→ Combine sorted runs into larger chunks.

Sorted Run

A run is a list of key/value pairs.

Key: The attribute(s) to compare to compute the sort order.

Value: Two choices

- Tuple (early materialization).
- Record ID (late materialization).

Early Materialization

K_1	<i><Tuple Data></i>
K_2	<i><Tuple Data></i>
⋮	

Late Materialization



2-Way External Merge Sort

We will start with a simple example of a 2-way external merge sort.

→ “2” is the number of runs that we are going to merge into a new run for each pass.

Data is broken up into **N** pages.

The DBMS has a finite number of **B** buffer pool pages to hold input and output data.

Simplified 2-Way External Merge Sort

Pass #0

- Read one page of the table into memory
- Sort page into a “run” and write it back to disk
- Repeat until the whole table has been sorted into runs

Pass #1,2,3,...

- Recursively merge pairs of runs into runs twice as long
- Need at least 3 buffer pages (2 for input, 1 for output)

Simplified 2-Way External Merge Sort

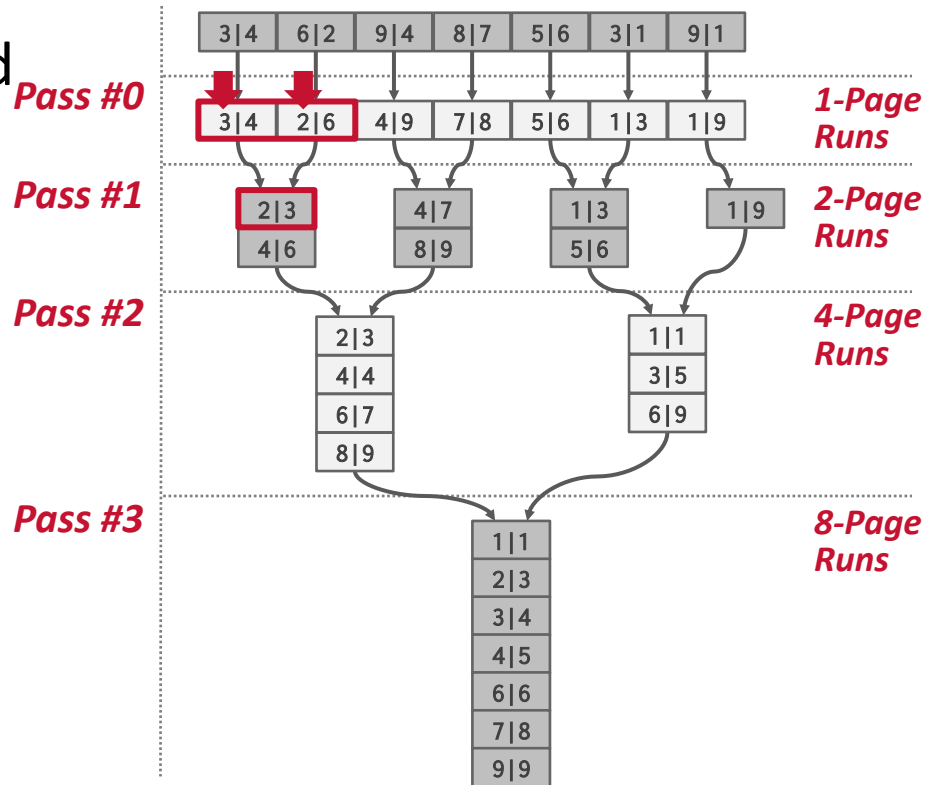
In each pass, the DBMS reads and writes every page in the file.

Number of passes

$$= 1 + \lceil \log_2 N \rceil$$

Total I/O cost

$$= 2N \cdot (\text{\# of passes})$$



Simplified 2-Way External Merge Sort

This algorithm only requires three buffer pool pages to perform the sorting ($B=3$).

→ Two input pages, one output page

But even if we have more buffer space available ($B>3$), it does not effectively utilize them if the worker must block on disk I/O...

General External Merge Sort

Pass #0

- Use B buffer pages
- Produce $\lceil N / B \rceil$ sorted runs of size B

Pass #1,2,3,...

- Merge $B-1$ runs (i.e., M-way merge)

Number of passes = $1 + \lceil \log_{B-1} \lceil N / B \rceil \rceil$

Total I/O Cost = $2N \cdot (\# \text{ of passes})$

Example

Determine how many passes it takes to sort 108 pages with 5 buffer pool pages: **$N=108$, $B=5$**

- **Pass #0:** **$\lceil N / B \rceil = \lceil 108 / 5 \rceil = 22$** sorted runs of 5 pages each (last run is only 3 pages).
- **Pass #1:** **$\lceil N' / B-1 \rceil = \lceil 22 / 4 \rceil = 6$** sorted runs of 20 pages each (last run is only 8 pages).
- **Pass #2:** **$\lceil N'' / B-1 \rceil = \lceil 6 / 4 \rceil = 2$** sorted runs, first one has 80 pages and second one has 28 pages.
- **Pass #3:** Sorted file of 108 pages.

$$1 + \lceil \log_{B-1} \lceil N / B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 1 + \lceil 2.229... \rceil = 4 \text{ passes}$$

Double Buffering Optimization

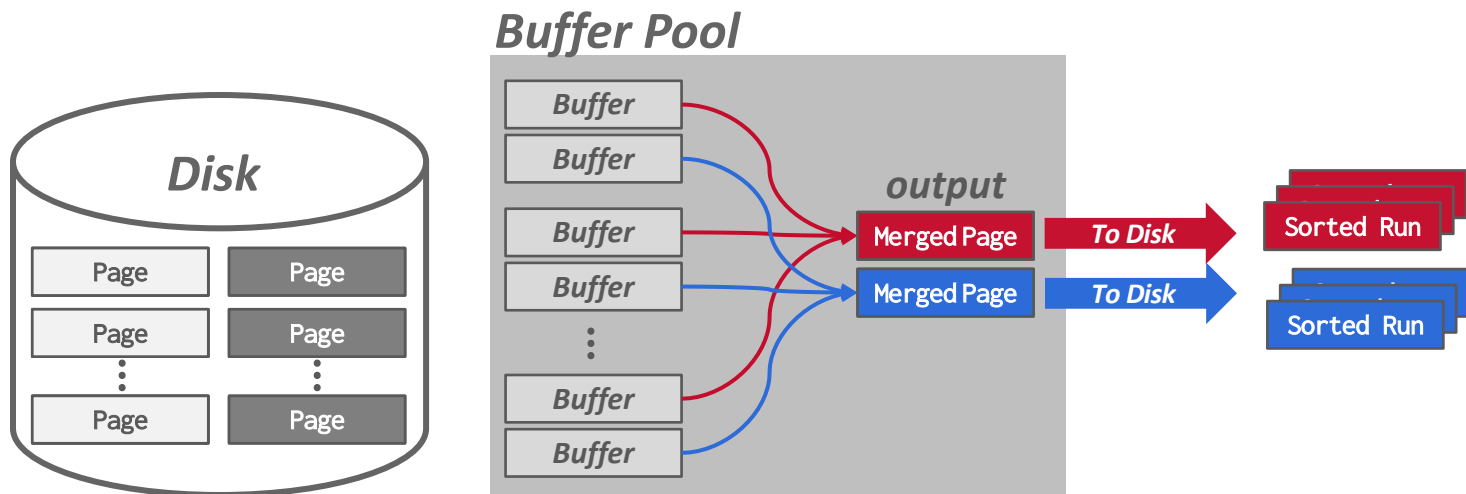
Prefetch the next run in the background and store it in a second buffer while the system is processing the current run.

→ Reduces the wait time for I/O requests at each step by continuously utilizing the disk.

Double Buffering

Prefetch next run in the background and store in a second buffer while processing the current run.

- Overlap CPU and I/O operations
- Reduces effective buffers available by half



Comparison Optimizations

Approach #1: Code Specialization

- Instead of providing a comparison function as a pointer to sorting algorithm, create a hardcoded version of sort that is specific to a key type.

Approach #2: Suffix Truncation

- First compare a binary prefix of long **VARCHAR** keys instead of slower string comparison. Fallback to slower version if prefixes are equal.

Using B+Trees For Sorting

If the table that must be sorted already has a B+Tree index on the sort attribute(s), then we can use that to accelerate sorting.

→ Some DBMSs support prefix key scans for sorting.

Retrieve tuples in desired sort order by simply traversing the leaf pages of the tree.

Cases to consider:

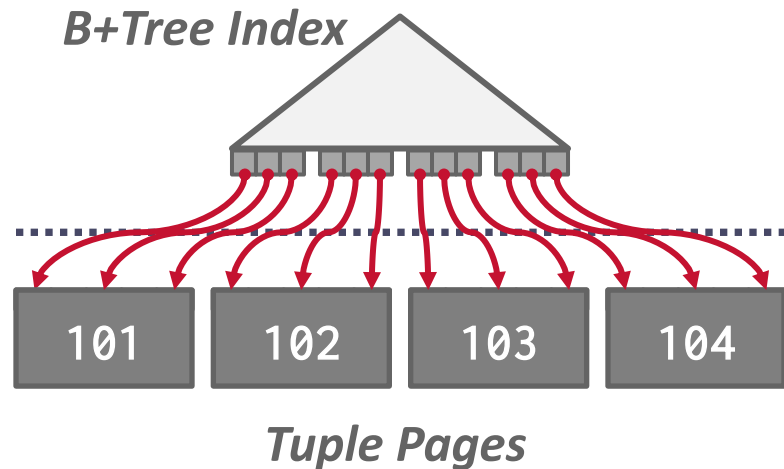
→ Clustered B+Tree

→ Unclustered B+Tree

Case 1: Clustered B+Trees For Sorting

Traverse to the left-most leaf page,
and then retrieve tuples from all leaf
pages.

This is always better than external
sorting because there is no
computational cost, and all disk
access is sequential.

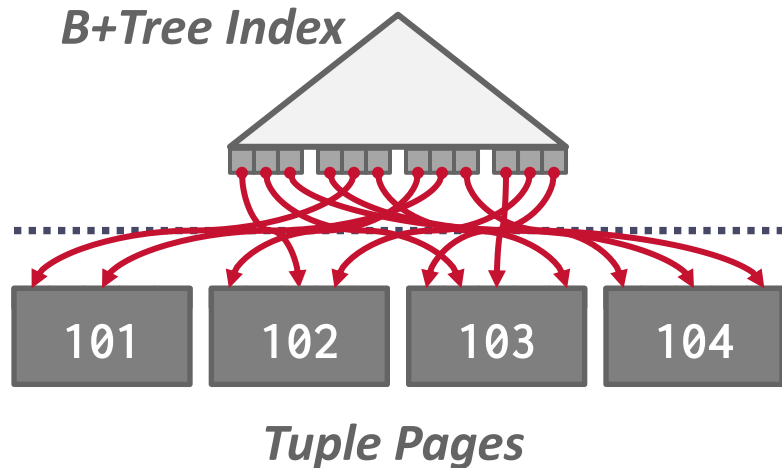


Case 2: Unclustered B+Trees For Sorting

Chase each pointer to the page that contains the data.

This is almost always a bad idea except for Top-N queries where N is small enough relative to total number of tuples in table.

→ In general, one I/O per data record.



Aggregations

Collapse values for a single attribute from multiple tuples into a single scalar value.

The DBMS needs a way to quickly find tuples with the same distinguishing attributes for grouping.

Two implementation choices:

- Sorting
- Hashing

Sorting Aggregation

```
SELECT DISTINCT cid
  FROM enrolled
 WHERE grade IN ('B','C')
 ORDER BY cid
```

enrolled (sid, cid, grade)

sid	cid	grade
53666	15-445	C
53688	15-721	A
53688	15-826	B
53666	15-721	C
53655	15-445	C


Filter

sid	cid	grade
53666	15-445	C
53688	15-826	B
53666	15-721	C
53655	15-445	C


**Remove
Columns**

cid
15-445
15-826
15-721
15-445


Sort

cid
15-445
15-445
15-721
15-826


**Eliminate
Duplicates**

Alternatives To Sorting

What if we do not need the data to be ordered?

- Forming groups in **GROUP BY** (no ordering)
- Removing duplicates in **DISTINCT** (no ordering)

Hashing is a better alternative in this scenario.

- Only need to remove duplicates, no need for ordering.
- Can be computationally cheaper than sorting.

Hashing Aggregate

Populate an ephemeral hash table as the DBMS scans the table. For each record, check whether there is already an entry in the hash table:

- **DISTINCT**: Discard duplicate
- **GROUP BY**: Perform aggregate computation

If everything fits in memory, then this is easy.

If the DBMS must spill data to disk, then we need to be smarter...

External Hashing Aggregate

Divide-and-conquer approach to computing an aggregation when data does not fit in memory.

Phase #1 – Partition

- Split tuples into buckets based on hash key
- Write them out to disk when they get full

Phase #2 – ReHash

- Build in-memory hash table for each partition and compute the aggregation

Phase #1 – Partition

Use a hash function h_1 to split tuples into partitions on disk.

- A partition is one or more pages that contain the set of keys with the same hash value.
- Partitions are “spilled” to disk via output buffers.

Assume that we have B buffers.

We will use $B-1$ buffers for the partitions and 1 buffer for the input data.

Phase #1 – Partition

```
SELECT DISTINCT cid
  FROM enrolled
 WHERE grade IN ('B','C')
```

enrolled (sid, cid, grade)

sid	cid	grade
53666	15-445	C
53688	15-721	A
53688	15-826	B
53666	15-721	C
53655	15-445	C

Filter

sid	cid	grade
53666	15-445	C
53688	15-826	B
53666	15-721	C
53655	15-445	C

⋮

Remove Columns

cid
15-445
15-826
15-721
15-445

⋮

B-1 partitions

h_1

15-445 15-445
15-445 15-312
15-312 15-445

15-826
15-210

⋮

15-721



Phase #2 – Rehash

For each partition on disk:

- Read it into memory and build an in-memory hash table based on a second hash function h_2 .
- Then go through each bucket of this hash table to bring together matching tuples.

This assumes that each partition fits in memory.

Phase #2 – Rehash

```
SELECT DISTINCT cid
  FROM enrolled
 WHERE grade IN ('B','C')
```

enrolled (sid, cid, grade)

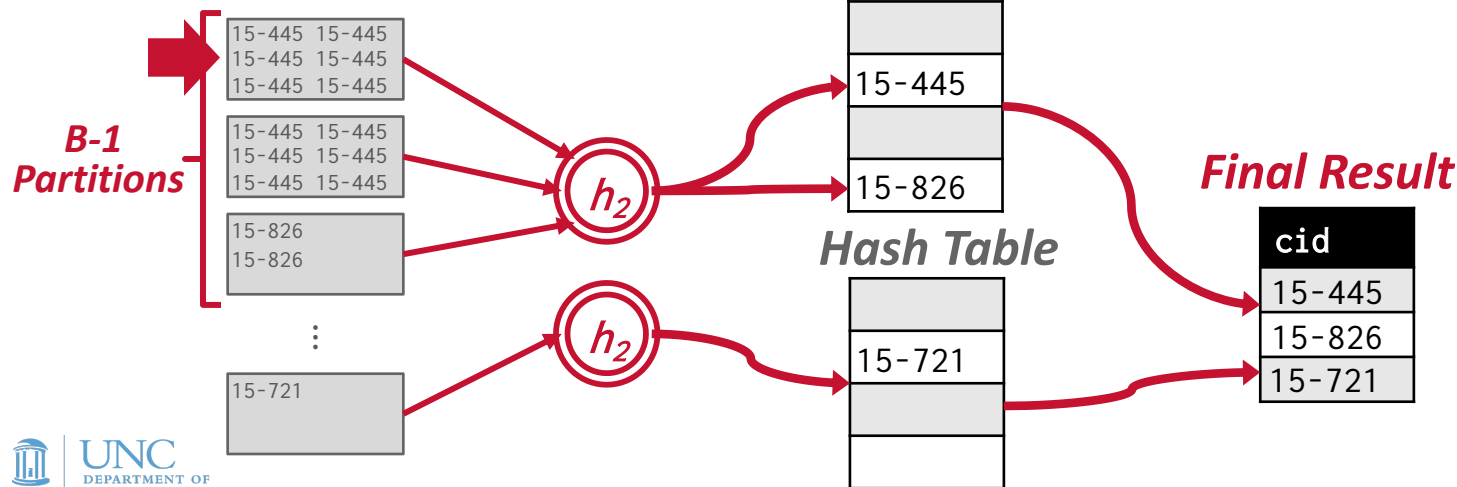
sid	cid	grade
53666	15-445	C
53688	15-721	A
53688	15-826	B
53666	15-721	C
53655	15-445	C

Phase #1 Buckets

Hash Table

Hash Table

Final Result



Hashing Summarization

During the rehash phase, store pairs of the form
(**GroupKey**→**RunningVal**)

When we want to insert a new tuple into the hash table as we compute the aggregate:

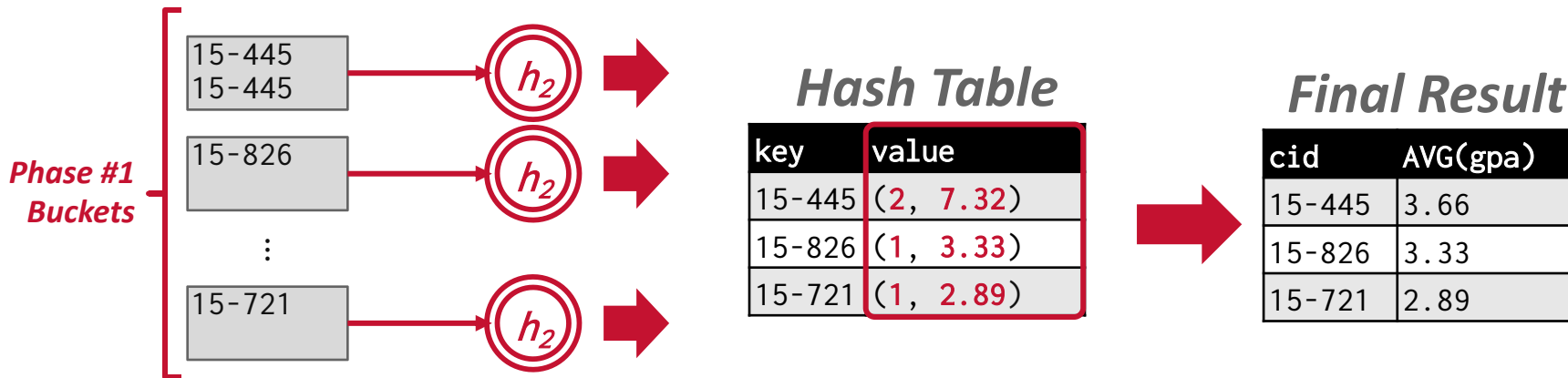
- If we find a matching **GroupKey**, just update the **RunningVal** appropriately
- Else insert a new **GroupKey**→**RunningVal**

Hashing Summarization

```
SELECT cid, AVG(s.gpa)
  FROM student AS s, enrolled AS e
 WHERE s.sid = e.sid
 GROUP BY cid
```

Running Totals

AVG(col) → (COUNT, SUM)
 MIN(col) → (MIN)
 MAX(col) → (MAX)
 SUM(col) → (SUM)
 COUNT(col) → (COUNT)



Conclusion

Choice of sorting vs. hashing is subtle and depends on optimizations done in each case.

We already discussed the optimizations for sorting:

- Chunk I/O into large blocks to amortize costs
- Double-buffering to overlap CPU and I/O

Next Class

Miscellaneous data structures, and how to make them thread-safe